

TASVIRLARNI TAHLIL QILISH VA ANGLASH · Amaliy mashg'ulot (Practice)

13-amaliy mashg'ulot. Kamerani kalibrlash va 3D rekonstruksiya

2026–2027 o'quv yili

Mavzu. Kamera modeli $P = K[R | t]$ va kalibrlash; epipolyar geometriya (F , essential matrix E); holatni tiklash va masshtab; triangulation va reprojection error; PnP va RANSAC; SfM va bundle adjustment; point cloud, ICP va NeRF.

Maqsadlar. Talaba: (1) proyeksiya, E , triangulation, reprojection error, RANSAC iteratsiyalari va ICP qadamini qo'lda hisoblaydi; (2) asosiy tasdiqlarni (epipolyar shart, masshtab noaniqligi, 4 yechim) keltirib chiqaradi; (3) OpenCV/NumPy/SciPy da ikki ko'rinishli zanjir, PnP, ICP va kichik bundle adjustment ni yozadi va natijani ma'lum haqiqiy qiymat (ground truth) bilan tekshiradi.

Baholash (100 ball). A bo'lim (1–11, qo'lda hisoblash): $11 \times 2 = 22$; B bo'lim (12–16, isbot va tushuncha): $5 \times 3 = 15$; C bo'lim (17–30, dasturlash): $14 \times 4,5 = 63$. Kod topshiriqlarida: to'g'ri ishlashi — 60%, chiqish formati va sonlar — 25%, izoh/tahlil — 15%. Sonlar kutilgan javobdan kichik farq qilishi mumkin (turli BLAS), lekin tartibi va xulosasi bir xil bo'lishi kerak.

Muhit. Python 3.11, `pip install -r requirements.txt: numpy==2.4.4, scipy==1.17.1, opencv-python-headless==4.13.0.92, scikit-learn==1.8.0, torch==2.14.0` (bu mavzuda kerak emas).

Barcha ma'lumotlar sun'iy (`seed=42`) — internet kerak emas. Faqat CPU. Kodni `python -W error::DeprecationWarning` bilan ishga tushiring — ogohlantirish chiqmasligi kerak.

Hujjatlar. https://docs.opencv.org/4.x/d9/d0c/group_calib3d.html,
https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html,
https://docs.opencv.org/4.x/da/de9/tutorial_py_epipolar_geometry.html,
<https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html>,
https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html,
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html>.

1. A bo'lim. Qo'lda hisoblash

Barcha javoblarni kamida 3 ta qiymatli raqam bilan yozing; oraliq qadamlarni ko'rsating.

- Proyeksiya.** $K = \begin{pmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{pmatrix}$, $R = I$, $t = (-0,2; 0,1; 1)^T$ m, $X = (0,6; -0,4; 4)^T$ m. Kamera koordinatalaridagi nuqta va (u, v) piksel koordinatalarini toping.
- Essential matrix.** $R = R_z(90^\circ) = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, $t = (1, 0, 0)^T$, $K = I$. (a) $E = [t]_{\times} R$ ni yozing; (b) singulyar qiymatlarini toping; (c) $X_1 = (1, 2, 5)^T$ uchun $\hat{x}_1, \hat{x}_2$ ni topib, $\hat{x}_2^T E \hat{x}_1 = 0$ ni tekshiring; (d) epipollar e_1 ($E e_1 = 0$) va e_2 ($E^T e_2 = 0$) ni toping.
- Triangulation.** $K = \begin{pmatrix} 500 & 0 & 250 \\ 0 & 500 & 250 \\ 0 & 0 & 1 \end{pmatrix}$, $P_1 = K[I | 0]$, $P_2 = K[I | (-1, 0, 0)^T]$. Moslik: $x_1 = (300, 275)$, $x_2 = (200, 275)$. DLT sistemasining 4×4 matritsasi A ni yozing va X ni toping. Natijani $Z = fb/d$ bilan tekshiring.
- Reprojection error.** 1-masaladagi K , $R = I$, $t = 0$; tiklangan nuqta $X = (-0,3; 0,45; 6)$, kuzatilgan nuqta $(279,0; 301,5)$. e ni toping; 1 px chegarasi bo'yicha xulosa qiling.
- Chuqurlik xatosi.** $f = 700$ px, $b = 0,2$ m, $\Delta d = 0,5$ px. $Z = 6$ m va $Z = 3$ m da ΔZ ; $Z = 6$ m da nisbiy xato va triangulation burchagi $2 \arctan(b/2Z)$; 1% nisbiy aniqlik uchun maksimal Z .
- RANSAC.** Inlierlar ulushi $w = 0,6$. 5-point ($s = 5$) va 8-point ($s = 8$) uchun $p = 0,99$ va $p = 0,999$ da minimal N ni toping (yuqoriga yaxlitlang). Xulosa?
- Sampson masofasi.** 2-masaladagi E va $\hat{x}_1 = (0,2; 0,4; 1)$; ikkinchi nuqta shovqin bilan $\hat{x}_2 = (-0,2; 0,23; 1)$ kuzatilgan. Epipolyar chiziq $l_2 = E \hat{x}_1$, qoldiq $\hat{x}_2^T E \hat{x}_1$ va Sampson masofasini toping; $f = 800$ px da pikselga o'tkazing.
- ICP qadami (2D).** $Q = \{(0, 0), (2, 0), (2, 1), (0, 1)\}$, $P = \{(0,3; -0,1), (2,2; 0,3), (1,9; 1,3), (-0,1; 0,9)\}$. (a) Har p_i uchun eng yaqin q ; (b) markazlar $\bar{p}, \bar{q}$; (c) markazlangan koordinatalarda $S_{xx} = \sum p_x q_x$, S_{yy} , $S_{xy} = \sum p_x q_y$, S_{yx} va $\theta = \text{atan2}(S_{xy} - S_{yx}, S_{xx} + S_{yy})$; (d) $t = \bar{q} - R(\theta)\bar{p}$; (e) qadamdan oldin va keyingi RMSE.
- Bundle adjustment o'lchami.** 30 kamera (har biri 6 parametr), 8000 nuqta, har nuqta o'rtacha 6 kadrda ko'rinadi. Noma'lumlar va qoldiqlar soni, Schur to'ldiruvchisi o'lchami, gauge erkinliklari soni.
- NeRF volume rendering.** Nur bo'ylab $\sigma = (0,5; 4; 1)$, $\delta = 0,4$, ranglar qizil, yashil, ko'k. α_i , T_i , og'irliklar $T_i \alpha_i$, $\hat{C}$ va $\sum T_i \alpha_i = 1 - T_4$ ni tekshiring. $L = 6$ da 3D nuqta uchun positional encoding o'lchami?
- Masshtab.** Ikki kadrda tiklangan sahnada ($\|t\| = 1$) uzunligi 0,25 m bo'lgan chizg'ich uchlari orasidagi masofa 0,08 birlik chiqdi. Masshtab s , metrik baza va tiklangan chuqurligi 2,4 birlik bo'lgan nuqtaning metrik chuqurligi?

2. B boʻlim. Isbot va tushuncha

12. $X_2 = RX_1 + t$ dan epipolyar shart $\hat{x}_2^T [t]_{\times} R \hat{x}_1 = 0$ ni keltirib chiqaring va E ning singulyar qiymatlari $(\|t\|, \|t\|, 0)$ ekanini isbotlang.
13. F nega 7, E nega 5 erkinlik darajasiga ega? 8-point algoritmda Hartley normallashi nima uchun kerak (sonli shartlanganlik nuqtai nazaridan)? Barcha nuqtalar bitta tekislikda boʻlsa nima boʻladi?
14. $E = U \text{diag}(1, 1, 0) V^T$ dan 4 ta (R, t) yechim qanday olinadi? Cheirality sharti bittasini qanday tanlaydi? Masshtab noaniqligini isbotlang: $\{sX_i\}$, st bir xil tasvir beradi.
15. $N \geq \ln(1-p)/\ln(1-w^s)$ formulasini keltirib chiqaring. Nega RANSAC dan keyin inlierlar boʻyicha qayta baholash kerak? Nega PnP essential matrix ga qaraganda aniqroq va masshtabni ham beradi?
16. ICP da maqsad funksiya har iteratsiyada oʻsmasligini isbotlang (ikki qadam: moslik va Kabsch). Nega ICP faqat lokal minimumga yaqinlashadi va amalda qanday boshlangʻich taxmin olinadi? Kabsch da $\det(VU^T) = -1$ tuzatishi nima uchun kerak?

3. C boʻlim. Dasturlash (NumPy / OpenCV / SciPy)

Har bir funksiya alohida yoziladi; `main()` ularni tartib bilan chaqirib, koʻrsatilgan formatda (`[raqam] ...`) chop etadi. Umumiy sunʼiy sahna (17–27-topshiriqlar shu kod bilan boshlanadi):

```
import numpy as np, cv2
cv2.setNumThreads(1); SEED = 42
K = np.array([[800., 0, 320], [0, 800, 240], [0, 0, 1]])
def rot_y(deg):
    a = np.deg2rad(deg); return np.array([[np.cos(a), 0, np.sin(a)], [0, 1, 0], [-np.sin(a), 0, np.cos(a)]])
def make_scene(n=200, noise=0.5, seed=SEED):
    rng = np.random.default_rng(seed)
    X = np.c_[rng.uniform(-2, 2, (n, 2)), rng.uniform(4, 10, n)] # (n,3), metr
    R, t = rot_y(7), np.array([-0.6, 0.05, 0.1])
    P1 = K @ np.hstack([np.eye(3), np.zeros((3, 1))]); P2 = K @ np.hstack([R, t[:, None]])
    x1 = project(P1, X) + rng.normal(0, noise, (n, 2))
    x2 = project(P2, X) + rng.normal(0, noise, (n, 2))
    return dict(X=X, R=R, t=t, P1=P1, P2=P2, x1=x1, x2=x2)
# burilish xatosi: arccos((tr(Ra^T Rb)-1)/2); yoʻnalish xatosi: arccos(ta.tb/|ta||tb|), gradusda
```

17. `project(P, X) -> (N,2)` ni yozing. `make_scene(noise=0)` da 2-kamera uchun `cv2.projectPoints` (`rvec=cv2.Rodrigues(R)[0]`) bilan solishtiring. *Chiqish:* [17] first point cam2 = (u, v) px; max|own - cv2| =
18. **Kalibrlash.** K_{true} : $f = 900$, $(c_x, c_y) = (640, 360)$, distorsiya $(k_1, k_2) = (-0.2; 0.05)$, tasvir 1280×720 . Shaxmat taxtasi 9×6 , katak 25 mm; 12 koʻrinish: `rvec=rng.uniform(-0.4,0.4,3)`, `tvec=[-0.1,-0.07,0.3]+rng.uniform(-0.05,0.05,3)`, `cv2.projectPoints + N(0; 0.2^2)` px shovqin (`rng=default_rng(42)`). `cv2.calibrateCamera(..., flags=cv2.CALIB_FIX_K3)`. *Chiqish:* [18] rms=... px fx=... fy=... cx=... cy=... k1=... k2=....
19. `eight_point(x1, x2) -> F` (Hartley normallashi, rank 2, $\|F\| = 1$) va `sampson(F, x1, x2) -> (N,)`. `make_scene(noise=0.5)` da haqiqiy $F = K^{-T}[t]_{\times} RK^{-1}$ va baholangan F uchun oʻrtacha Sampson masofasi, $\sigma_3(F)$. *Chiqish:* [19] Sampson mean: `F_true=... px`, `F_8pt=... px`; `sigma3(F_8pt)=...`
20. 19-topshiriqdagi F dan $E = K^T F K$ (normallangan). Singulyar qiymatlar nisbati σ/σ_1 ; E ni $U \text{diag}(\bar{\sigma}, \bar{\sigma}, 0) V^T$ bilan “tozalang” va $\min \|E_c \mp E_{\text{true}}\|_F$ ni toping (ikkalasi ham normallangan). *Chiqish:* [20] `sv(E)/sv1 = [...]`; `cleaned E: min||E_c -/+ E_true||_F = ...`
21. `decompose_E(E) -> [(R,t)]*4` ($\det U = \det V = 1$ ga keltiring). Shovqinsiz sahnada har nomzod uchun ikkala kamera oldidagi ($Z > 0$) nuqtalar sonini (22-topshiriqdagi DLT bilan) toping; eng yaxshisini haqiqiy R va `cv2.recoverPose` bilan solishtiring. *Chiqish:* [21] `in-front counts = [...]`; `best=...`; `dR(best,true)=...`; `dR(best,recoverPose)=...`; `dt=...`
22. `triangulate_dlt(P1, P2, x1, x2) -> (N,3)` (SVD). `noise=0.5` da haqiqiy P_1, P_2 bilan `cv2.triangulatePoints` dan farq va 3D xato (oʻrtacha, mediana). *Chiqish:* [22] `max|own - cv2| = ... m`; `3D error mean=... m` `median=... m`.
23. **Use-case:** ikki koʻrinishli rekonstruksiya. `noise=0.5`; `cv2.setRNGSeed(42)`; `cv2.findEssentialMat(x1, x2, K, cv2.RANSAC, 0.999, 1.0) -> cv2.recoverPose -> cv2.triangulatePoints`. Masshtabni haqiqiy bazadan oling ($s = \|t\|/\|\hat{t}\|$). Inlierlar, ΔR , Δt (yoʻnalish), 3D xato medianasi, 2-kameradagi oʻrtacha reprojection error. *Chiqish:* [23] `inliers=...` `dR=...` deg `dt=...` deg `|t|=...` `scale=...` va `3D error median=... m`, `mean reproj(cam2)=... px`.
24. **Masshtab noaniqligi.** Shovqinsiz sahnada $X \rightarrow 10X$, $t \rightarrow 10t$; ikkala kamerada tasvirlarning maksimal farqi. *Chiqish:* [24] `scale x10: max image difference = ... px`.
25. **Chuqurlik xatosi.** `make_scene(n=3000, noise=1.0)`, haqiqiy P_1, P_2 bilan DLT. $Z \in [4; 5.5)$, $[5.5; 7.5)$,

- [7,5;10] guruhlarda oʻrtacha 3D xato (m va %); oxirgi/birinchi nisbatni Z^2 qonuni ($8,75^2/4,75^2$) bilan solishtiring. *Chiqish:* [25] `Z in [lo,hi]: n=... mean 3D error=... m (... %) (3 qator) va [25] ratio last/first observed=..., Z^2 law=...`
- 26. RANSAC ishonchliligi.** `noise=0.5; outlier ulushi 0,10,20,40%: rng=default_rng(43)` (har ulush uchun qaytadan), `idx=rng.choice(200,k,replace=False), x2[idx]=rng.uniform([0,0],[640,480],(k,2))`. Bar-cha nuqtalar boʻyicha 8-point ($E = K^T FK \rightarrow \text{recoverPose}$) va 23-topshiriqdagi RANSAC zanjiri uchun ΔR . *Chiqish:* [26] `outliers=...%: 8-point dR=... deg | RANSAC dR=... deg, inliers=... (4 qator)`.
- 27. PnP.** `noise=1.0; 2-kamera kuzatuvlarining 40 tasini (default_rng(44), yuqoridagi usul) tasodifiy nuqtaga almashtiring. cv2.setRNGSeed(42); cv2.solvePnP(Ransac(X, x2, K, None, iterationsCount=200, reprojectionError=3.0, confidence=0.999, flags=cv2.SOLVEPNP_EPNP), so'ng inliarlarda cv2.solvePnPRefineLM. Chiqish: [27] ok=... inliers=... (outliers rejected ../40) dR=... deg dt=... m.`
- 28. NeRF nuri.** `posenc(p, L) va render_ray(sigma, color, delta) -> (C, w).` Nur: `z=np.linspace(2,4,65)`, nuqtalar — oraliq oʻrtalari, $\delta = \text{np.diff}(z)$; $|z - 3| \leq 0,5$ da $\sigma = 20$, aks holda 0; rang (0,2; 0,6; 1). $\sum w, \hat{C}$, kutilgan chuqurlik $\sum w_i z_i / \sum w_i$; $L = 10$ va $L = 4$ da encoding oʻlchami. *Chiqish:* [28] `opacity sum w=... C=[...] expected depth=...; posenc dims L=10:... L=4:...`
- 29. Use-case: point cloud ICP.** Sirt: `u,v=default_rng(42).uniform(-1,1,(2,1500))`, $z = 0,3 \sin 2u \cos 2v + 0,2u^2$ (Q). `rng=default_rng(45); $\theta \in \{15^\circ, 60^\circ\}$ uchun $R_t = R_z(\theta)R_y(\theta/3)$, $t_t = (0,1; -0,05; 0,08)$, $P=(Q-t_t)@R_t+ \text{rng.normal}(0,0.005,Q.\text{shape})$. kabsch(P,Q) va icp(P,Q, iters=50, tol=1e-10) (cKDTree, RMSE oʻzgarishi $< \text{tol}$ da toʻxtash). Qaysi holatda ICP lokal minimumda qoladi? Chiqish: [29] init rot=.. deg: iters=.. RMSE a -> b m, dR=... deg, |dt|=... m (2 qator).`
- 30. Use-case: kichik bundle adjustment.** `rng=default_rng(42)` (tartib: nuqtalar, kuzatuv shovqini, kamera va nuqta gʻalayoni): 80 nuqta (sahnadagi kabi), K (pinhole), 3 kamera, parametrlar (r, t) (Rodrigues): $(0,0)$ (mahkamlangan), $((0,5^\circ,0), (-0,5; 0; 0,05))$, $((0,-6^\circ,0), (0,6; 0,05; 0))$. Kuzatuvlar $+ \mathcal{N}(0; 0,5^2)$; boshlangʻich taxmin: kameralarga $\mathcal{N}(0; [0,01 \times 3, 0,05 \times 3])$, nuqtalarga $\mathcal{N}(0; 0,1^2)$. `scipy.optimize.least_squares(method='trf', jac_sparsity=..., x_scale='jac', ftol=xtol=1e-10)`. *Chiqish:* [30] `unknowns=... residuals=...; mean reproj a px -> b px (nfev=...)`.